

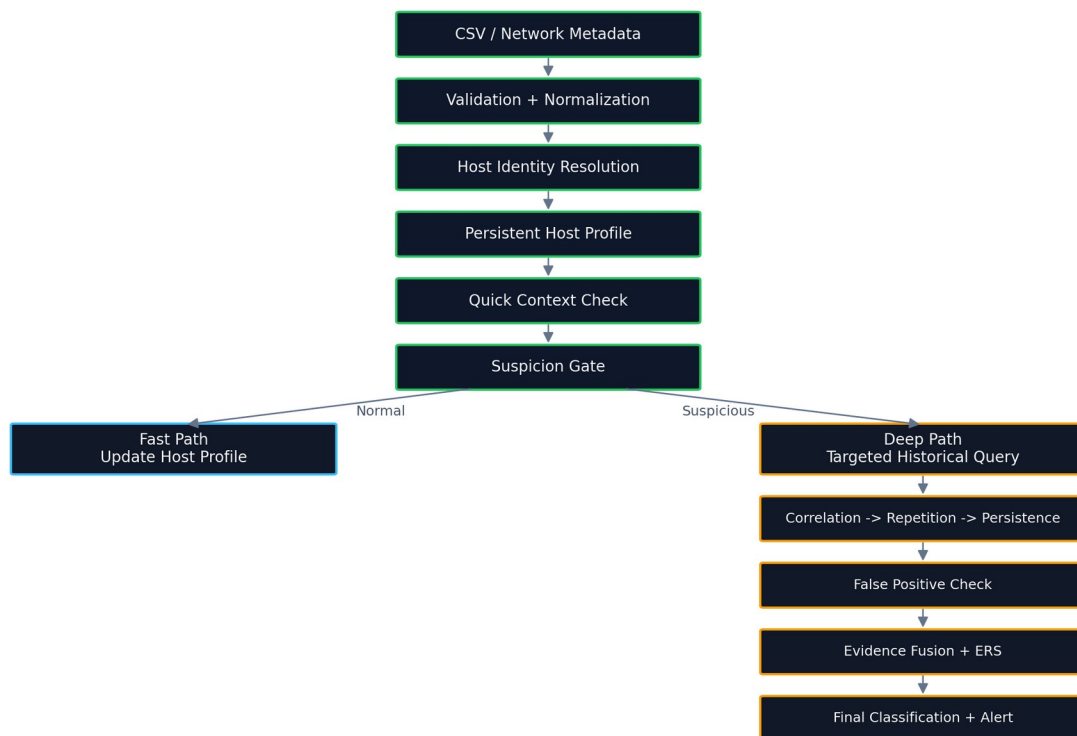
LEAKSIGNAL

PT 1

Prototype 1 - Detailed Build Blueprint

Enterprise slow data exfiltration detection using persistent host profiles, targeted historical context, behavioural analysis, and an explainable Exfiltration Risk Score.

LeakSignal Prototype 1 - Detection Flow



Document purpose: define exactly what Prototype 1 must prove, how it will be built, how each component works, how it will be tested, and what counts as complete.

1. Prototype 1 Mission

Prototype 1 objective

Prove one complete detection loop: individually harmless-looking network events should become suspicious only when LeakSignal builds enough host-specific evidence over time.

Prototype 1 is not the final enterprise product. It is the smallest technically credible version that proves the main LeakSignal idea end-to-end.

- Accept structured network metadata from a controlled CSV dataset.
- Resolve each event to a stable host identity.
- Maintain a persistent behavioural profile for every host.
- Perform a fast context check on every event.
- Trigger deeper historical queries only when an event is suspicious.
- Correlate related events by host, destination, timing, repetition, and persistence.
- Suppress known legitimate activity such as scheduled backups.
- Calculate an explainable Exfiltration Risk Score (ERS).
- Classify the final state as Normal, Monitor, Suspicious, or Possible Slow Data Exfiltration.
- Display the evidence clearly in a web dashboard.

Core rule

Normal events stay on the fast path. Suspicious events earn deeper investigation.

This is the central efficiency and scalability rule of Prototype 1. The system should not perform expensive historical analysis for every event.

2. What Prototype 1 Will and Will Not Build

Included in PT 1	Not included yet
CSV/network metadata ingestion	Live packet sniffing or deep packet inspection
Host-specific behavioural baseline	Full SIEM replacement
Quick anomaly/context checks	Firewall blocking or automated containment
Targeted historical lookup	API-key/file-content scanning
Repetition and persistence detection	Malware detection
False-positive check	Enterprise SSO/user management
Isolation Forest as supporting signal	Multiple ML models or LLM chatbot
Explainable ERS and dashboard	Massive threat-intelligence feeds

3. Three Detection Scenarios for Prototype 1

The first prototype should stay selective. It will prove three enterprise-relevant scenarios rather than trying to detect every possible data-leak case.

Scenario A - Slow-Drip Exfiltration

A finance workstation sends small outbound transfers to the same unusual external destination at night. Each transfer is too small to justify a critical alert by itself, but the pattern repeats across multiple days.

```
FIN-PC-07
Normal outbound: 30-50 MB/day
Attack pattern:
Day 1 - 5 MB - 02:15 AM - 185.44.22.91
Day 2 - 7 MB - 02:42 AM - same destination
Day 3 - 6 MB - 02:11 AM - same destination
Day 4 - 8 MB - 02:37 AM - same destination
```

Expected behaviour: Day 1 may only cause Monitor. As repetition and persistence accumulate, ERS rises until the host becomes Suspicious or Possible Slow Data Exfiltration.

Scenario B - New Destination + Off-Hours Activity

An HR workstation normally communicates with known services during business hours. It suddenly sends data to a first-seen destination at 02:15 AM.

```
HR-PC-02
Normal hours: 09:00-18:00
Known destinations: Microsoft 365, internal HR systems
New event: 12 MB - 02:15 AM - never-before-seen destination
```

Expected behaviour: initial suspicion increases because the time and destination are abnormal. Deeper analysis is triggered only if the quick context score crosses the prototype threshold.

Scenario C - Legitimate Scheduled Backup

A backup server sends hundreds of megabytes every night. Volume and timing look suspicious, but the destination and schedule are known and historically stable.

```
BACKUP-SERVER-01
02:00 AM daily
400-600 MB outbound
Destination: approved backup service
Pattern: stable for weeks
```

Expected behaviour: false-positive logic reduces the risk because the pattern is approved, scheduled, and consistent with the host profile. Final state should remain Normal or low Monitor, not Critical.

4. Phase 1 - Build the Controlled Dataset

The dataset is the foundation of Prototype 1. We need enough normal history to build host baselines and enough injected suspicious behaviour to demonstrate slow-drip correlation.

Parameter	Prototype 1 target	Why
Hosts	20-30	Enough behavioural variety without making debugging difficult.
History	7 days initially	Enough to demonstrate daily timing patterns and persistence.
Events	5,000-20,000	Large enough for meaningful baselines, small enough for rapid iteration.
Attack hosts	2-3	Allows focused, explainable storylines.
False-positive host	At least 1	Proves the system does not equate unusual traffic with exfiltration.

Recommended event schema

Field	Purpose
event_id	Unique event reference.
timestamp	When activity occurred.
host_id	Stable host identity.
src_ip	Current source IP context.
dst_ip	External/internal destination.
dst_port	Destination service context.
protocol	TCP/UDP/etc.
bytes_sent	Primary outbound-volume signal.
bytes_received	Inbound context.
duration	Connection duration.
destination_category	Approved cloud, internal, unknown, backup, etc.

```
timestamp,host_id,src_ip,dst_ip,dst_port,protocol,bytes_sent,duration
2026-08-20 10:32:17,FIN-PC-07,10.0.0.17,142.250.1.1,443,TCP,154000,18
2026-08-21 02:15:05,FIN-PC-07,10.0.0.17,185.44.22.91,443,TCP,5242880,47
```

Dataset generation strategy

1. Generate normal business-hour traffic for all hosts.
2. Give departments different patterns: Finance, HR, Development, Backup/Infrastructure.
3. Inject slow-drip events into one finance host across multiple nights.
4. Inject a first-seen/off-hours pattern into one HR or developer host.
5. Create one legitimate scheduled backup pattern and mark its destination as approved.
6. Keep ground-truth labels separately for evaluation; do not feed those labels into detection logic.

5. Phase 2 - Event Ingestion, Validation and Normalization

Prototype 1 begins with CSV upload. Live feeds are intentionally deferred. FastAPI receives the file, validates each row, normalizes fields, and stores valid events.

Validation check	Example
Required fields present	timestamp, host_id, destination, bytes_sent
Timestamp valid	Parse to one canonical timezone/format
IP/host values usable	Reject malformed addresses or missing host identifiers
Byte counts numeric and non-negative	5,242,880 bytes is valid; negative values are not
Traffic direction known	Prototype focuses on outbound activity
Protocol normalized	TCP/tcp/Tcp -> TCP

Why this matters

Bad input creates bad baselines. Validation must happen before any anomaly or risk logic. A malformed timestamp can falsely create an off-hours signal; a duplicated event can falsely create repetition.

6. Phase 3 - Stable Host Identity

LeakSignal should not treat an IP address as the permanent identity of a machine. DHCP and network changes can alter IPs. Prototype 1 therefore uses a stable host_id and stores IP as context.

```
host_id: FIN-PC-07
hostname: FINANCE-LT-07
department: Finance
current_ip: 10.0.0.17
first_seen: 2026-08-20
last_seen: 2026-08-27
risk_state: Monitor
```

Minimum hosts table

Field	Meaning
host_id	Stable primary identity
hostname	Human-readable machine name
department	Useful context for later baselines
current_ip	Latest IP observed
first_seen / last_seen	Host lifecycle context
risk_state	Latest final classification

7. Phase 4 - Persistent Host Profile

Each host gets one logical profile in the database. This is the short-term memory LeakSignal uses for fast checks without rereading the entire event history.

Profile component	Example for FIN-PC-07
Normal outbound/day	43 MB
Normal active hours	08:45-19:10
Typical destinations	Microsoft 365, GitHub, approved company cloud
Average transfer size	1.8 MB
Night-traffic ratio	Very low
Recent destinations	Last N known destinations
Recent ERS	18
Alert history	No previous critical alerts

8. Phase 5 - Behavioural Baseline

The baseline answers a simple question: what is normal for this specific host? LeakSignal should avoid one global threshold because a backup server and an HR laptop naturally behave differently.

- Average bytes sent per hour and per day.
- Normal active-hour distribution.
- Typical destination set and destination frequency.
- Average and percentile transfer size.
- Night-time and weekend traffic ratio.
- Unique destinations per day.
- Rolling mean and standard deviation.
- EWMA for recent behavioural changes.

Example

```
FIN-PC-07
Mean outbound: 42 MB/day
Std. deviation: 8 MB/day
Normal hours: 09:00-18:30
Current outbound: 91 MB/day
Observed change: significantly above host baseline
```

Important

Volume deviation is evidence, not a verdict. A large transfer can be legitimate; LeakSignal combines multiple signals before classifying exfiltration.

9. Phase 6 - Quick Context Check

Every new event receives a cheap initial evaluation using the persistent host profile. This is the fast path that makes continuous monitoring practical.

Quick signal	Question	Example
Off-hours	Is this time unusual for this host?	02:15 AM vs normal 09:00-18:30
First-seen destination	Has this host contacted this destination before?	185.44.22.91 has 0 prior contacts
Rare destination	Is the destination uncommon for this host?	Seen once in 7 days
Transfer deviation	Is this event larger/smaller than normal?	9 MB vs typical 1.8 MB
Frequency deviation	Are transfers happening unusually often?	Every 45 minutes overnight

Fast-path decision

```
No meaningful suspicion -> update profile -> store event -> continue
Suspicious context -> trigger targeted deep analysis
```


10. Phase 7 - Suspicion Gate

Prototype 1 uses a lightweight pre-score to decide whether an event deserves a deeper database query. These values are implementation placeholders and must be tuned during testing.

Initial signal	Example PT1 weight
First-seen destination	+20
Off-hours activity	+15
Transfer above baseline	+15
Rare destination	+10

Temporary suspicion score < 25 -> fast path
Temporary suspicion score >= 25 -> deep context path

11. Phase 8 - On-Demand Deep Context Gathering

This is the main efficiency improvement in the final LeakSignal architecture. The database is not asked for everything the host has ever done. It retrieves only history related to the suspicious event.

Current event:
Host: FIN-PC-07
Destination: 185.44.22.91
Time: 02:15 AM
Transfer: 5 MB

Targeted query:
Same host + same/related destination + similar time window + recent lookback period

Query dimension	Prototype 1 example
Host	FIN-PC-07
Destination	185.44.22.91
Time window	01:00-04:00
Lookback	Previous 7 days
Transfer similarity	Small repeated outbound sessions

Result example

The single 5 MB event looked harmless. Targeted history reveals 5 MB, 7 MB, 6 MB, and 8 MB transfers to the same destination on four different nights. That is the slow-drip pattern LeakSignal is designed to expose.

12. Phase 9 - Event Correlation

Correlation converts isolated events into a developing pattern. Prototype 1 groups relevant events using a small set of explainable keys.

- Same stable host identity
- Same or related destination
- Similar time-of-day
- Similar transfer size/range
- Recurring interval or frequency
- Pattern across multiple days

FIN-PC-07

- 185.44.22.91 | 02:15 | 5 MB

- 185.44.22.91 | 02:42 | 7 MB

- 185.44.22.91 | 02:11 | 6 MB

- 185.44.22.91 | 02:37 | 8 MB

13. Phase 10 - Repetition and Persistence

These are separate concepts and should be calculated separately.

Concept	Question	Example
---------	----------	---------

Repetition	Did this behaviour happen more than once?	Four similar transfers
Persistence	Did it continue across meaningful time?	Same behaviour across four nights

Prototype features

- Repeated transfer count
- Number of affected days
- Average interval between events
- Night-time repetition count
- Destination repetition
- Persistence duration in hours/days

14. Phase 11 - False-Positive Control

Before a high-risk verdict, LeakSignal asks whether the same evidence could be explained by legitimate automation. Prototype 1 must include at least one case where suspicious-looking traffic is correctly downgraded.

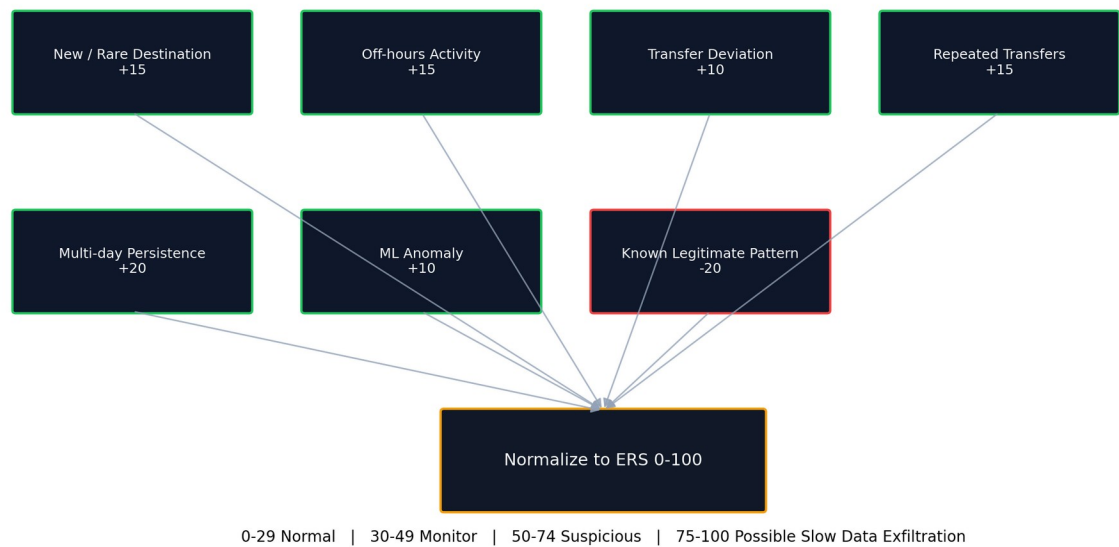
Legitimate context	How PT1 handles it
Approved backup destination	Risk reduction / known allow context
Scheduled nightly job	Expected timing pattern
Approved cloud synchronization	Known destination category
Stable repeated automation	Low novelty despite high volume
Maintenance/update window	Optional approved time context

BACKUP-SERVER-01
Large transfer + night activity = suspicious signals
BUT approved destination + stable daily schedule = legitimate pattern
Final result: Normal / low Monitor

15. Phase 12 - Exfiltration Risk Score (ERS) v1

ERS combines evidence into one explainable score. Prototype 1 starts with transparent rule weights. Later prototypes can tune them using validation data.

Prototype 1 - Exfiltration Risk Score (ERS) v1



Signal	Illustrative PT1 weight
New destination	+15
Rare destination	+10
Off-hours activity	+15
Volume deviation	+10

Repeated transfers	+15
Multi-day persistence	+20
Isolation Forest anomaly	+10
Known legitimate pattern	-20

Initial classification bands

ERS	Classification
0-29	Normal
30-49	Monitor
50-74	Suspicious
75-100	Possible Slow Data Exfiltration

Tuning note

These thresholds and weights are not architecture constants. They are Prototype 1 starting values and must be adjusted during implementation based on false positives, detection behaviour, and demo data.

16. Phase 13 - Isolation Forest as a Supporting Signal

Isolation Forest is added only after the deterministic pipeline works. It should strengthen or weaken evidence, not independently declare data exfiltration.

Candidate feature	Purpose
bytes_sent	Outbound magnitude
hour_of_day	Timing abnormality
destination_rarity	Novelty of destination
session_duration	Connection behaviour
event_frequency	Burst/repetition behaviour
outbound_ratio	Traffic direction imbalance
unique_destination_count	Destination change behaviour

Isolation Forest -> anomaly score
 Anomaly score -> one ERS component
 Final verdict -> combined evidence, never ML alone

Why this matters

A model can call a valid backup unusual. Explainable rules, persistent context, and false-positive checks keep the final decision grounded.

17. Phase 14 - Final Decision Engine

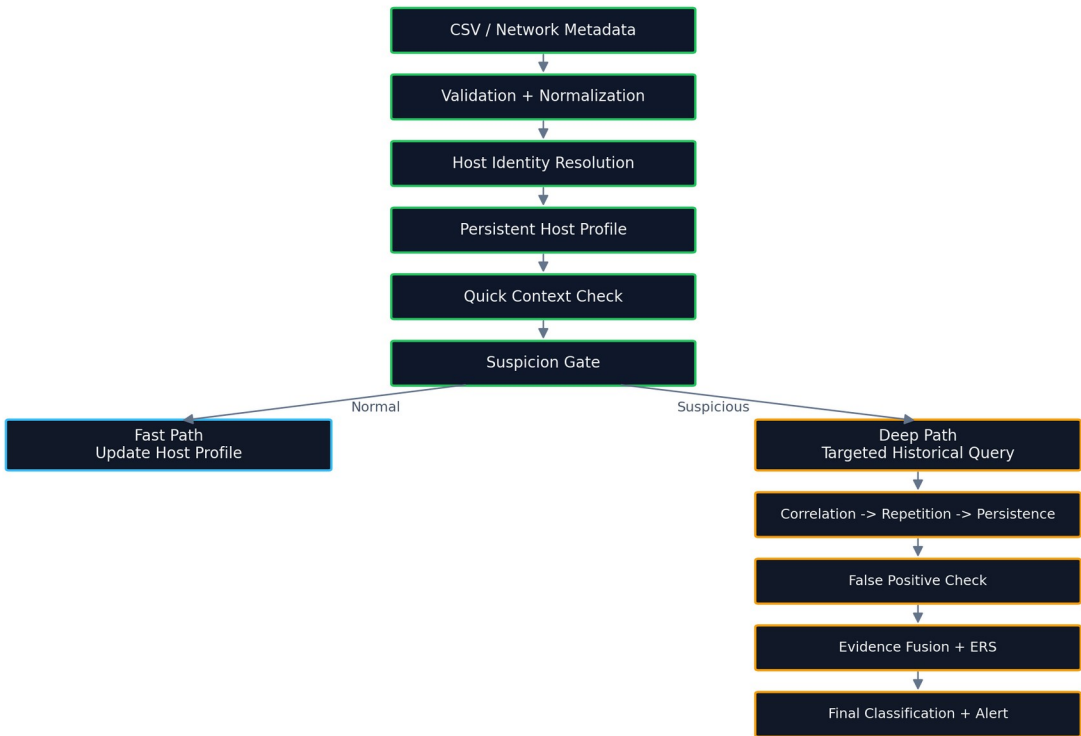
Every investigated host/event pattern ends in one of four states. The dashboard must show the state and the evidence that caused it.

State	Meaning
Normal	Behaviour matches expected host context.
Monitor	Some unusual signals exist but evidence is weak or isolated.
Suspicious	Multiple correlated signals indicate meaningful risk.
Possible Slow Data Exfiltration	Strong repeated/persistent evidence supports a likely slow-leak pattern.

HOST: FIN-PC-07
 ERS: 91/100
 Classification: POSSIBLE SLOW DATA EXFILTRATION
 Evidence:
 - First-seen destination
 - Off-hours activity
 - Repeated small transfers
 - Pattern persisted 4 days
 - Strong deviation from host baseline
 - Isolation Forest anomaly

18. Prototype 1 Technical Architecture

LeakSignal Prototype 1 - Detection Flow



Recommended stack

Layer	Technology	Responsibility
Frontend	Next.js / React	Dashboard, host views, alerts, evidence timeline
Backend	Python + FastAPI	Ingestion, APIs, orchestration
Detection	Pandas / NumPy	Feature calculations, baselines, windows
ML	scikit-learn Isolation Forest	Supporting anomaly score
Database	PostgreSQL / Supabase	Hosts, events, profiles, alerts, ERS
Hosting	Vercel + Render + Supabase	Free prototype deployment path

19. Backend Codebase Blueprint

```
backend/
|- ingestion/
|  |- csv_loader.py
|  `-- validator.py
|- profiles/
|  |- host_identity.py
|  `-- host_profile.py
|- baseline/
|  `-- baseline_engine.py
|- detection/
|  |- quick_checks.py
|  |- suspicion_gate.py
|  |- deep_context.py
|  |- correlation.py
|  |- repetition.py
|  |- persistence.py
|  `-- false_positive.py
|- risk/
|  `-- ers.py
|- ml/
|  `-- isolation_forest.py
|- api/
|  |- hosts.py
|  |- alerts.py
|  |- upload.py
|  `-- dashboard.py
|- main.py
```

Module responsibility rule

Keep detection stages separate. A quick-check module should not secretly perform correlation; ERS should not query raw history. Clear boundaries make the prototype easier to debug and explain to judges.

20. Database Blueprint

Table	Minimum contents
hosts	host_id, hostname, department, current_ip, first_seen, last_seen, risk_state
network_events	event_id, host_id, timestamp, src_ip, dst_ip, port, protocol, bytes_sent, bytes_received, duration
host_profiles	baseline metrics, active-hour profile, destination summary, transfer behaviour, recent risk state
destinations	destination identity/category, first_seen, last_seen, approved flag
risk_scores	host_id, ERS, evidence components, timestamp
alerts	alert_id, host_id, classification, severity, explanation, status

21. FastAPI Endpoint Plan

Endpoint	Purpose
----------	---------

POST /api/upload	Upload Prototype 1 CSV dataset
POST /api/events	Future-friendly single/batch event ingestion
GET /api/dashboard	Summary metrics
GET /api/hosts	Host list + latest risk
GET /api/hosts/{id}	Host profile and current state
GET /api/hosts/{id}/timeline	Behaviour/risk timeline
GET /api/alerts	Alert feed
GET /api/alerts/{id}	Evidence and explanation for one alert

22. Prototype 1 Dashboard

Do not build a giant SOC interface. Prototype 1 needs only three screens, each answering a specific question.

Page A - Overview

```
LEAKSIGNAL
Monitored Hosts: 24
Critical Risk Hosts: 1
Suspicious Hosts: 2
Alerts Today: 4

HIGH-RISK HOSTS
FIN-PC-07    ERS 91    Possible Slow Data Exfiltration
DEV-PC-03    ERS 68    Suspicious
HR-PC-11     ERS 57    Suspicious
```

Question answered: Which company assets need attention right now?

Page B - Host Investigation

```
FIN-PC-07
ERS 91 / 100
Status: Possible Slow Data Exfiltration
Normal outbound: 43 MB/day
Current: 106 MB/day
New destination: 185.44.22.91
Activity: 02:00-03:30 AM
Repeated: 4 consecutive nights
```

Question answered: How did this host move from normal to risky?

Page C - Alert Evidence

```
WHY LEAKSIGNAL FLAGGED THIS
+ First-seen destination
+ Off-hours transfer
+ Repeated small outbound sessions
+ Multi-day persistence
+ Host baseline deviation
+ ML anomaly support

FALSE-POSITIVE CHECK
No approved backup/sync pattern found
```

Question answered: Why should an analyst trust this alert?

23. Testing Strategy

Prototype 1 is successful only if it detects the injected storylines and avoids obvious false positives. Testing should be deterministic before it becomes visual.

Test	Expected result
Normal business-hour traffic to known destinations	Normal
One small off-hours transfer to new destination	Monitor, not automatically critical

Same pattern repeating across several days	Risk escalates
Repeated small transfers to same rare destination	Correlation/persistence evidence increases
Large approved nightly backup	Suppressed/downgraded by false-positive logic
Isolation Forest anomaly without supporting evidence	No critical verdict by ML alone
Malformed event row	Rejected/flagged during validation

24. Prototype 1 Evaluation Metrics

The first prototype does not need a research-paper evaluation, but it should measure enough to prove behaviour and support later tuning.

Metric	Why measure it
Detection of injected slow-drip cases	Does the core concept work?
False-positive count	Does legitimate activity trigger too many alerts?
Time to process normal event	Does the fast path stay lightweight?
Percentage of events triggering deep queries	Are deep queries selective?
ERS progression over attack days	Does evidence accumulate correctly?
Explanation completeness	Can every score be tied to visible evidence?
Database query size/time	Is targeted history materially smaller than broad history?

25. Recommended 12-Day Build Roadmap

Prototype 1 - Recommended 12-Day Build Roadmap

Goal: One reliable end-to-end demo that proves slow-drip detection and explainable escalation.



Day	Primary output
1	Freeze data schema, host types, ground-truth scenarios.
2	Generate normal + attack + backup demo dataset.
3	Build validation, normalization, host identity.
4	Persistent host profile + baseline engine.
5	Quick context checks + suspicion gate.
6	Targeted historical query + correlation + repetition/persistence.
7	False-positive logic + ERS v1.
8	Isolation Forest integration and feature tuning.

9	FastAPI endpoints + database integration.
10	Overview, host detail, alert evidence dashboard.
11	End-to-end tests, thresholds, bug fixing.
12	Demo polish, screenshots, fallback dataset, final rehearsal.

26. Final Prototype 1 Demo Story

The demo should tell one simple story rather than clicking through every feature.

7. Open the LeakSignal overview and show that most hosts are Normal.
8. Load or process the controlled dataset.
9. Open FIN-PC-07 and show its established normal baseline.
10. Show Day 1: a small 5 MB transfer to a new destination at 02:15 AM. LeakSignal marks it Monitor rather than overreacting.
11. Advance to Day 2 and Day 3. The same destination and timing recur; targeted historical analysis finds the pattern.
12. Show the repetition and persistence evidence increasing.
13. On Day 4, show ERS reaching the high/critical band and the final classification becoming Possible Slow Data Exfiltration.
14. Open the evidence panel and explain every contributing signal.
15. Then open BACKUP-SERVER-01 and show that a much larger nightly transfer is correctly recognized as a known scheduled backup.
16. End with the message: LeakSignal does not panic over one anomaly; it builds evidence over time.

Ideal closing line

LeakSignal detects data theft that is intentionally too slow and subtle to look dangerous when each event is viewed alone.

27. Risks and Prototype 1 Fallbacks

Risk	Fallback / control
Baseline too weak for new hosts	Use seeded normal-history period and mark cold-start confidence.
Too many false positives	Increase reliance on destination history, scheduling context, and persistence before high severity.
Isolation Forest unstable	Keep ML as optional supporting evidence; deterministic pipeline still works.
Cloud backend sleeps/fails during demo	Maintain local backend mode and preloaded dataset.
ERS feels arbitrary	Show every component weight and tune using known ground-truth cases.
Dashboard consumes too much time	Prioritize host detail + alert explanation over decorative widgets.

28. Definition of Done - Prototype 1

Prototype 1 is complete only when every item below is demonstrably working.

Status	Requirement
[]	CSV network metadata is accepted and validated.
[]	Events are normalized and stored.
[]	Stable host identity works independently of IP changes.
[]	Persistent host profiles are created and updated.
[]	Host-specific baselines are available.
[]	Off-hours and new/rare destination checks work.
[]	Suspicion gate separates fast path from deep path.
[]	Deep historical queries retrieve only relevant context.
[]	Related events are correlated.
[]	Repetition and persistence are separately measured.
[]	Known scheduled backup is suppressed/downgraded.
[]	ERS is calculated from visible evidence.
[]	Four final classifications work.
[]	Isolation Forest contributes as a supporting signal.
[]	Dashboard shows overview, host investigation, and alert evidence.
[]	End-to-end slow-drip demo works reliably.

29. What Prototype 1 Proves

- LeakSignal can distinguish host-specific normal behaviour from developing abnormal behaviour.
- A single harmless-looking event does not automatically become a critical alert.
- Suspicious events trigger deeper context only when needed.
- Repeated small transfers become meaningful when correlated across time.
- Legitimate scheduled behaviour can be recognized before the final verdict.
- The final risk decision can be explained in terms an analyst can inspect.

30. Transition to Prototype 2

Prototype 2 should not redesign the detection logic. It should productize the proven engine: better real-time ingestion, richer analyst workflow, stronger destination context, improved scaling, more realistic enterprise data sources, and better operational monitoring.

Prototype 1 final principle

Identify the host -> understand its normal behaviour -> quickly evaluate the event -> gather deeper relevant history only when needed -> correlate over time -> detect repetition and persistence -> filter legitimate behaviour -> calculate explainable risk -> decide.

Source basis

This build blueprint is based on the finalized LeakSignal detection-logic architecture supplied for this project. The architecture is treated as fixed for Prototype 1; thresholds, lookback periods, transfer definitions, and ERS weights remain implementation-level tuning parameters.